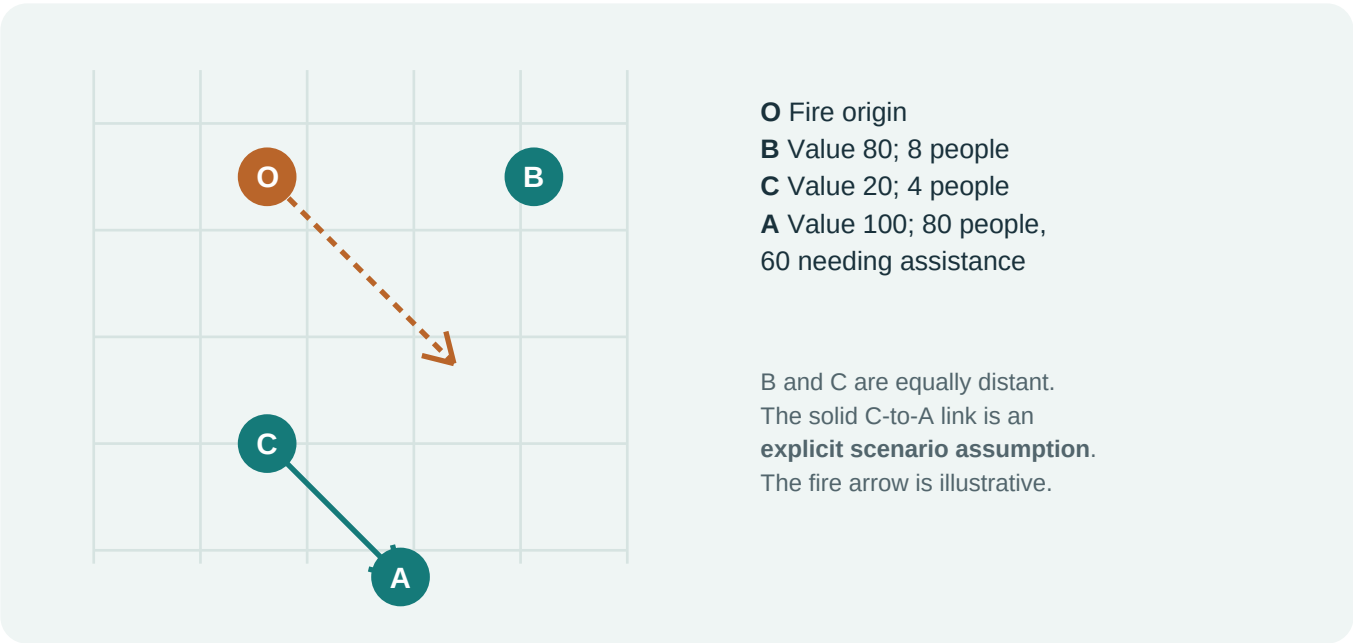


Two priorities. Two algorithms.

Who should be contacted first, and which sequence of response actions best meets the supplied objectives?

A location ranking cannot represent an action that unlocks or benefits another location. The prototype separates **independent contact scores** from a **constrained action-sequence search**.



Output	Base fixture result
Contact order	A > B > C
Response sequence	C > A
Direct-value greedy baseline	B > C
Covered assisted-person units	Exact: 60; greedy: 0

Interpretation: C comes first because the fixture explicitly requires it before assisting A. The algorithm does not infer that firefighters at a nearby building can stop a fire. All coordinates, times and effects here are synthetic.

Rank locations independently.

Contact priority is an explainable heuristic; it does not determine a crew itinerary.

Fixed normalisation and explicit policy

$P = 1 / (1 + \text{distance} / 200 \text{ m})$
 $N = \min(\text{people} / 100, 1)$
 $V = \min(\text{value} / 100, 1)$
 $H = \min(\text{assisted people} / 100, 1)$
Contact score = 0.45 P + 0.20 N + 0.20 V + 0.15 H

Location	Distance (m)	People / assisted	Value	Score
A	316.2	80 / 60	100	0.624
B	200.0	8 / 0	80	0.401
C	200.0	4 / 0	20	0.273

Weights and normalisation scales are prototype configuration, not validated emergency policy. Equal scores use stable asset IDs. Unknown distance, people, assisted count or value produces a separate review record with a null score; unknown never becomes zero.

Static input contract

Input	Required meaning
Location	Stable ID, local metric coordinates, distance, people, assisted people, value, impact deadline.
Action	Site, duration, completion deadline, prerequisites, required capabilities.
Benefit edge	Target asset, coverage fraction [0,1], confirmation and evidence or labelled assumption.
Travel leg	Directed end-to-end travel minutes and optional availability cutoff. Missing leg means unavailable.
Crew / scenario	Start node, capability tags, planning horizon and nonnegative time buffer.

Compare complete feasible sequences.

Exact enumeration for one crew and at most eight actions. No GPS shortcut or assumed causal propagation.

Feasibility before benefit

An action requires its prerequisites, crew capabilities and an available directed travel leg. Arrival must precede the leg cutoff. Completion plus the configured buffer must fit the work-site deadline, action deadline and planning horizon. A target receives benefit only before its own deadline. Inclusive comparisons tolerate only 1e-9 minutes of arithmetic noise; the scenario buffer is separate.

Count each asset once

coverage(asset) = maximum confirmed, on-time effect from completed actions.

A 0.6 effect followed by an 0.8 effect gives 0.8 coverage, not 1.4. No transitive effects are invented. Unknown target inputs or unconfirmed effects receive no credit and create review items.

Objective, compared in order

1. Maximise covered assisted-person units.
2. Then maximise covered total-person units.
3. Then maximise covered asset-value units.
4. Then deliver each benefit category earlier, minimise completion time and action count, and break remaining ties by action ID.

Each total is $\text{sum}(\text{asset amount} \times \text{coverage})$. Assisted people are included in total people; these are separate priority dimensions, not added counts. Earlier delivery minimises completion time weighted by incremental benefit. This life-first ordering is an explicit prototype policy and needs analyst review.

Search procedure

Evaluate the current sequence, including the empty sequence.

For each unused action: check prerequisites and timing; extend if feasible.

Update coverage and incremental-benefit timing; recurse.

Retain the best sequence and the best alternative for every first action.

Zero-immediate-benefit actions are retained because they can unlock A. Worst-case search is factorial; eight actions have at most 109,601 partial sequences including the empty one. Inputs over eight actions are rejected, not silently truncated or reported as optimal. Optimality applies only to this supplied static model.

04 / EXECUTABLE CASES

Test the counterexamples too.

14 labelled cases run from the same model and produce the results below. All expectations passed.

Case / change	Exact order	Assisted units
C unlocks timely assistance at A	C > A	60
Declared C-to-A protection, no implicit propagation	C > B	60
A directly accessible: bypass unnecessary C action	A > C	60
No people at A and B has a tight deadline	B > C	0
C unreachable; do not invent a straight-line shortcut	B	0
Crew lacks assisted-evacuation capability	B > C	0
A deadline expires before the C-to-A sequence completes	B > C	0
Unknown occupancy is review, not zero or credited benefit	B > C	0
Nearby A receives no credit from an unconfirmed effect	B > C	0
Overlapping effects use maximum coverage, not addition	C	48
Zero-benefit C still unlocks A	C > A	60
C access closes before traversal completes	B	0
Three-minute buffer makes the A window infeasible	B	0
All work-site deadlines expired	No feasible action	0

"Covered units" are a model accounting measure under supplied assumptions, not a prediction of people saved. Unserved assets and review questions remain in the output even when another feasible action is selected.

Why C first is conditional.

Results are generated from the executable fixture rather than entered by hand.

Plan	Action timing	Assisted / total / value
Exact	C: 1-3 min; A: 4-7 min	60 / 84 / 120
Greedy	B: 1-3 min; C: 6-8 min	0 / 12 / 100

In the base fixture, C finishes at minute 3 and A at minute 7, before A's minute-9 deadline. Choosing B first leads to C finishing at minute 8; A would finish at minute 12 and is infeasible. If A is directly accessible, the algorithm instead chooses A first. If C is blocked or the crew lacks the capability needed at A, it reports the limitation and evaluates remaining actions.

Verification evidence

Test report: **99 tests, 0 failures/errors**. The report builder reads the supplied JUnit file; rerun tests before regenerating a release report.

Checks include stable ties, reverse input order, deadline equality, buffers, unavailable legs, capabilities, partial overlap, unknown counts, dependency cycles, invalid/non-finite values, duplicate IDs and the action-count limit. An independent permutation oracle compares small generated problems with the exact planner.

Algorithm references

[Google OR-Tools: Vehicle Routing Problem with Time Windows](#) describes the use of travel-time matrices and time windows. This prototype uses its own small exhaustive search, not OR-Tools.

[Google OR-Tools: Penalties and Dropping Visits](#) explains optional visits when all locations cannot be served. Here, unserved locations are explicit and the objective uses separate benefit priorities. Neither source validates the wildfire assumptions or policy weights.

A static prototype with clear limits.

Implementation: `codex/static-priorities` | Project owner for coordination: `@mirrdj`

Run from the worktree root

```
uv venv
uv pip install -e ".[dev,report]"
.venv/bin/python scripts/static_priorities.py
make static-priorities
.venv/bin/python -m pytest -q --junitxml=data/priority-tests.xml
make priority-report
```

Files to inspect

fixtures/static_priority.json - editable locations, actions and directed travel inputs.

fireline/contact_priority.py - independent contact scores.

fireline/response_priority.py - sequence search and myopic comparator.

reports/static-priority-results.json - scenario results, coverage and alternatives.

tests/test_static_priorities.py - executable behaviours and oracle checks.

What the algorithm cannot establish

The geometry is a synthetic local-metre layout; the southeast arrow does not estimate fire arrival. Deadlines, durations, access and benefit effects must be supplied. The code does not model smoke, fire intensity, suppression effectiveness, changing routes, safe crew egress, return to base, multiple crews, partial evacuation capacity or live weather. A completed action is assumed to deliver its declared persistent effect. Maximum coverage is a conservative overlap rule, not a simulation of distinct groups being rescued.

The result is a proposal for analyst review under these assumptions. Unknowns can change the answer; a mathematical optimum over the supplied inputs does not establish field safety. The existing live engine and UI are not automatically wired to this CLI experiment.

Next integration step

Map real location snapshots into this static input contract, with provenance and confirmed action relationships. Then add snapshot versioning and re-planning that preserves completed and assigned work. Larger problems and multiple crews need a scalable constrained solver and explicit resource/egress modelling before operational evaluation.